

KEEP - PPO Trading Hyperparameters - Kimi

=== "ENT_COEF", "downside_penalty", "LR", "NUM_EPOCHS", "NUM_STEPS", "MINI_BATCH_SIZE", "EVAL_FREQ", "PATIENCE", "WARMUP_EPOCHS" === i use daily OHLCV data to train a PPO RLVR agent for trading. recomend values for these parameters. ask if you need more info

Edit

Copy

Share

I need a few more details to give you well-grounded recommendations:

1. **How many assets** are in your universe? (single stock, 10 stocks, 500?)
2. **What's your action space?** (discrete: buy/hold/sell, or continuous position sizing?)
3. **Observation features:** raw OHLCV only, or do you include technical indicators / market context?
4. **Training data length** in years/days?
5. **Hardware constraint:** GPU available? Roughly how much memory?
6. **Reward:** PnL-based, Sharpe-based, or something custom?

In the meantime, here are **reasonable starting points** for a typical daily OHLCV single-stock trading setup:

Table

Copy

Parameter	Value	Rationale
ENT_COEF	0.01–0.05	Start at 0.01; increase if policy collapses to deterministic actions, decrease if exploration is too random
downside_penalty	0.5–2.0	1.0 is neutral; >1 penalizes drawdowns more aggressively (asymmetric risk aversion)
LR	1e-4 to 3e-4	Lower than typical RL because financial time series are noisy; use 3e-4 if features are normalized
NUM_EPOCHS	10–20	PPO's inner-loop epochs per batch; higher = more stable but risk overfitting to mini-batch
NUM_STEPS	256–2048	For daily data, 256–512 steps per rollout is common; scale with episode length
MINI_BATCH_SIZE	64–256	Must divide NUM_STEPS × num_envs; 64 for single env, 256+ for vectorized

Parameter	Value	Rationale
EVAL_FREQ	every 5–10 updates	Or every 1–2k steps; frequent enough to catch divergence early
PATIENCE	10–20 eval periods	Roughly 50–200 updates before early stopping
WARMUP_EPOCHS	2–5	Short warmup since PPO is relatively stable; mainly to let value network initialize

Key tuning notes for financial RL:

- NUM_STEPS should ideally span **multiple market regimes** (e.g., 512 days $\approx$ 2 years) so the agent sees both bull and bear within a rollout
- If using **multiple parallel envs** (highly recommended), multiply NUM_STEPS by env count for total batch size; keep mini-batch at 64–256
- ENT_COEF often needs **annealing** (decay from 0.05 $\rightarrow$ 0.001) as training progresses
- downside_penalty > 1.0 helps if your agent takes excessive leverage during drawdowns

Let me know the missing details and I can refine these significantly.

How many assets are in your universe = 800, What's your action space = 14, Observation features = 35, Training data length = 19 years, Validation data = 5 years, Test data = 5 years, Hardware = GPU 25 GB, Reward = Sharpe. Would these information change your recommended parameter, explain

Edit

Copy

Share

Yes, these details change the recommendations substantially. Here's the updated reasoning and values:

Key Implications of Your Setup

Table

Copy

Factor	Impact
800 assets	Large universe $\rightarrow$ need bigger batches for stable gradient estimates; action space complexity scales with cross-sectional signals
14 discrete actions	Moderate entropy needed to prevent premature convergence; action space isn't tiny but not huge
35 features	Relatively compact observation space; no need to shrink LR for high-dimensional inputs

Factor	Impact
19 years training (~4,750 days)	Massive data; risk of overfitting to noise if epochs too high; long rollouts feasible
Sharpe reward	Noisy, path-dependent signal → needs more samples per update to stabilize variance
25 GB GPU	Comfortable; can run 8–16 parallel envs with 800-asset observations

Revised Recommendations

Table

Copy

Parameter	Previous	Revised	Rationale
ENT_COEF	0.01–0.05	0.03 → anneal to 0.005	14 actions need exploration early, but Sharpe reward will eventually drive convergence; decay prevents random trading
downside_penalty	0.5–2.0	1.5–2.5	Sharpe already penalizes volatility, but with 800 assets the agent may load up on correlated downside risk; asymmetric penalty adds guardrail
LR	1e-4–3e-4	2e-4	Sweet spot: 35 features aren't high-dim, but 800-asset value function is large; too low and convergence is glacial on 19 years
NUM_EPOCHS	10–20	5–8	Critical change. With 4,750 days and 8+ parallel envs, you have enormous data per update. More epochs = severe overfitting to batch noise
NUM_STEPS	256–2048	512–1024	Must span regimes (your 5-year validation/test periods suggest you care about regime robustness). 1024 days ≈ 4 years, captures cycle variation
MINI_BATCH_SIZE	64–256	256–512	With 8 envs × 1024 steps = 8,192 samples per update, mini-batch of 512 gives 16 gradient steps — stable and efficient
EVAL_FREQ	5–10 updates	Every 2–3 updates	Sharpe is noisy; frequent eval on 5-year validation (your true test of generalization) catches overfitting early
PATIENCE	10–20 evals	15–25 evals	Tighter patience given eval frequency; ~30–75 updates before stopping
WARMUP_EPOCHS	2–5	1–2	With 5–8 total epochs, warmup is a smaller fraction; mainly just 1 epoch to stabilize value net init

Additional Architectural Suggestions

Table

Copy

Consideration	Recommendation
Parallel envs	Run 8–16 environments (different asset subsets or shuffled start dates). Total batch = <code>num_envs × NUM_STEPS</code> . This is the single biggest stability win for 800 assets.
Value network size	Use larger value net than policy (e.g., policy: 256→128, value: 512→256). Sharpe estimation is harder than action selection.
Feature whitening	Z-score normalize features per asset or cross-sectionally; RL is sensitive to scale.
Entropy annealing schedule	Linear decay: $0.03 \times (1 - \text{progress})$ or exponential $0.03 \times 0.995^{\text{update}}$.
Validation metric	Track out-of-sample Sharpe on the 5-year val set, not just episode return. Early stop when val Sharpe peaks — training Sharpe will keep rising while val Sharpe collapses (classic overfit).
Action masking	If some actions are invalid (e.g., shorting restricted assets), mask logits. With 800 assets, hard constraints prevent destabilizing gradients.

The Biggest Shift: NUM_EPOCHS Down, Batch Scale Up

Your instinct might be "more data = train longer per batch." **Opposite.** With 19 years of daily data, the signal-to-noise is low and temporal correlations are strong. Running 20 epochs on a batch of 1024×8 samples means seeing the same noisy Sharpe estimates 20 times — the policy memorizes spurious patterns. **5–8 epochs with massive parallel batches** generalizes better.

Want me to suggest a specific training loop structure or PPO variant (e.g., recurrent for temporal dependencies, or separate asset-specific vs. cross-sectional encoders)?

``Revised Recommendations`` i am trying to learn PPO RL. please explain how these parameters are used. explain it to a beginner

Edit

Copy

Share

I'll explain each PPO parameter from the ground up, building intuition for how they work together in your trading setup.

The Big Picture: What PPO Does

PPO (Proximal Policy Optimization) is like training a trader by:

1. **Letting them trade for a while** (collect experience)
2. **Watching a recording** of those trades (a "batch" of data)
3. **Giving feedback**: "When you saw this pattern, buying was good — but don't change your strategy too drastically"

The parameters control *how long* they trade, *how much* feedback they get, and *how aggressively* they adjust.

The Parameters Explained

NUM_STEPS = 512–1024

"How many days of trading before we review the tape?"

- Your agent trades for 512–1024 days, then you pause and analyze that chunk
 - **Why this matters**: Financial markets have "regimes" (bull, bear, sideways). If **NUM_STEPS** is too small (e.g., 64 days), your agent only sees one market type per review and learns fragile rules. 1024 days $\approx$ 4 years — enough to see multiple cycles.
 - **Analogy**: A basketball coach reviewing 4 seasons of tape vs. just 2 games
-

NUM_EPOCHS = 5–8

"How many times do we re-watch the same tape?"

- After collecting 1024 days of trades, you show the agent that *same batch* 5–8 times to learn from it
 - **Why low is better for you**: With 19 years of data, each batch is huge. Re-watching 20 times = overfitting to random noise ("that Tuesday in March 2008 felt special"). 5–8 times = extract the real pattern, move on.
 - **Analogy**: Re-reading a textbook chapter 5 times vs. 50 times. After a point, you're memorizing page numbers, not concepts.
-

MINI_BATCH_SIZE = 256–512

"How many trading days do we discuss at once?"

- You don't analyze all 1024 days at once. You break them into chunks of 256–512 days for gradient updates
- **Why this matters**:
 - Too small (32): Updates are noisy — like judging a trader's skill from 1 random week
 - Too large (2048): You need monster GPU memory; also less frequent updates = slower learning
- **Your 25GB GPU**: 512 is comfortable. With 8 parallel traders (see below), total batch = 8,192 days, split into 16 mini-batches of 512

LR (Learning Rate) = $2e-4$

"How big a step do we take when adjusting strategy?"

- After feedback, the agent updates its neural network weights. LR = step size.
 - **$2e-4 = 0.0002$** : Small, cautious steps
 - **Why not bigger**: Trading signals are weak and noisy. Big steps = "I saw one good trade, now I ALWAYS do that!" → instability
 - **Why not smaller**: With 19 years of data, you'd train forever
 - **Analogy**: Turning a ship's wheel. Too fast = zigzag. Too slow = miss the harbor.
-

ENT_COEF = 0.03 → 0.005

"How much do we reward trying new things?"

- "Entropy" = randomness/exploration. High entropy = agent tries diverse actions. Low = agent sticks to what it thinks is best.
 - **Schedule**: Start at 0.03 (explore widely among 14 actions), decay to 0.005 (exploit what works)
 - **Why decay**: Early training = agent knows nothing, needs to experiment. Later = should refine, not randomly trade.
 - **Without it**: Agent discovers "buy on Tuesdays works once" and does it forever, missing better strategies.
 - **Analogy**: A chef trying random ingredients early, then perfecting a recipe later.
-

downside_penalty = 1.5–2.5

"How much worse is losing money than making it?"

- Standard PPO treats +\$100 and −\$100 as symmetric. This parameter makes losses hurt more.
 - **1.5 = losses feel 1.5× worse than gains feel good**
 - **Why > 1 for you**: With 800 assets, the agent might find a "strategy" that wins big 90% of the time but crashes catastrophically 10%. Sharpe already penalizes volatility, but this adds extra guardrails against tail risk.
 - **Analogy**: A fund manager who gets fired for −20% drawdowns but only mildly praised for +20% returns.
-

EVAL_FREQ = Every 2–3 updates

"How often do we check if the agent is actually getting better?"

- After every 2–3 batches (each batch = 1024 days × 8 envs), pause training and test on **validation data** (your 5-year holdout)
 - **Why frequent:** Sharpe is noisy. Training might look good (agent memorized 2008 patterns) while validation Sharpe collapses.
 - **Analogy:** A student doing practice tests every week, not just at the end of the semester.
-

PATIENCE = 15–25 evals

"How many bad practice tests before we stop studying?"

- If validation Sharpe doesn't improve for 15–25 consecutive evaluations, stop training.
 - **Prevents overfitting:** Training Sharpe can rise forever as the agent memorizes noise. Validation Sharpe peaks, then drops.
 - **Analogy:** If your mile time isn't improving after 20 workouts, you're probably overtraining — rest.
-

WARMUP_EPOCHS = 1–2

"How many gentle passes before serious training?"

- First 1–2 epochs, the value network (which estimates "how good is this situation?") learns gently without aggressive policy updates
 - **Why:** The value network needs a rough idea of "what's good" before the policy can learn from it. Like calibrating a scale before weighing gold.
 - **Low for you:** With 5–8 total epochs, 1–2 is enough. More would waste precious learning time.
-

How They Connect: A Training Loop Walkthrough

plain

Copy

```
Step 1: Initialize agent (random trader)
      ↓
Step 2: Run 8 traders in parallel for 1024 days each
      → Collect 8,192 days of (observation, action, reward) data
      ↓
Step 3: Warmup: Gently train value network for 1–2 epochs
      ↓
Step 4: Main training: Re-watch the 8,192 days for 5–8 epochs
      → Split into mini-batches of 512 days
      → Update policy with LR=2e-4, entropy=0.03
      → Penalize downside losses 1.5× more
      ↓
Step 5: Every 2–3 loops, test on 5-year validation data
```

```
→ If Sharpe improved: keep going
→ If no improvement for 15-25 tests: STOP (early stopping)
↓
Step 6: Entropy decays 0.03 → 0.005 over time
↓
Repeat Step 2-6 until patience runs out
```

The "Why" for Your Specific Setup

Table

Copy

Your Constraint	Parameter Impact
800 assets	Need parallel envs (8+) and bigger mini-batches for stable gradients across many securities
14 actions	Moderate entropy needed; too low and agent converges to 1–2 actions, too high and it trades randomly
35 features	Not huge, so LR can be standard (2e-4); no need for tiny LR like with 1000+ features
19 years data	Low NUM_EPOCHS (5–8) — data abundance means overfitting risk, not data scarcity
Sharpe reward	Noisy, path-dependent → frequent eval (EVAL_FREQ every 2–3) and patience to catch overfitting
25GB GPU	Enables 8 parallel envs + 512 mini-batches; would struggle with 2048+ mini-batches

Common Beginner Mistakes to Avoid

Table

Copy

Mistake	Why It Hurts
High NUM_EPOCHS (20+)	Agent memorizes noise in 19 years of data; validation Sharpe tanks
Huge NUM_STEPS (4096+)	One batch spans 16 years — you're training and validating on overlapping periods
No entropy decay	Agent stops exploring; misses better strategies in 14-action space
No downside penalty	Agent loves high-volatility strategies that look good in Sharpe until they blow up
Ignoring validation	Training Sharpe rises while true generalization collapses

Want me to visualize the training loop flow, or explain how the **value network** and **policy network** interact differently?

Want me to visualize the training loop flow, or explain how the value network and policy network interact differently?==Yes, please

Edit

Copy

Share

Here's the visual breakdown of how the two networks interact during training:

The Analogy: Actor vs. Critic

Table

Copy

	Policy Network (The Trader)	Value Network (The Coach)
Job	Decides what action to take	Judges how good a situation is
Input	Same market features (35)	Same market features (35)
Output	Probability distribution over 14 actions	Single number: expected future Sharpe
Learning signal	"Was my action better than expected?"	"How close was my prediction to reality?"
Loss function	PPO clipped objective	Mean Squared Error (regression)
Network size	Smaller (256→128)	Larger (512→256) — harder job!

Why They Need Each Other

Without the Value Network, the Policy only sees raw rewards. A +0.8 Sharpe day looks great — but if *every* day in that market regime averages +1.2 Sharpe, the Policy actually underperformed. It would learn to repeat a **bad** action.

The Value Network provides the "baseline." It says: *"In this market state, a typical trader would expect +1.2 Sharpe."*

Then the **Advantage** = Actual – Baseline = 0.8 – 1.2 = **–0.4** (bad action!)

The Policy learns from **Advantage**, not raw reward. This is why the Value Network is essential.

The Warmup Phase Explained

WARMUP_EPOCHS = 1–2 exists because the Value Network starts as a **random guesser**. If the Policy tries to learn from random baselines, it gets nonsense feedback.

- **Epoch 1–2:** Only the Value Network updates. It watches trades and learns to roughly predict Sharpe.
- **Epoch 3+:** Both networks update together. The Policy now has a semi-reliable critic.

Think of it as: the coach watches a few games before giving the player serious feedback.

Why Different Network Sizes?

- **Policy:** "Given this state, which of 14 actions?" → Classification-like, easier
- **Value:** "Given this state, what's the exact expected Sharpe over the next N days?" → Regression, harder, needs more capacity

With 800 assets, the Value Network must implicitly model cross-asset correlations to predict portfolio Sharpe. That's why it gets the bigger brain.

The PPO "Clip" — What It Actually Does

The Policy wants to increase probability of good actions. But without a guardrail, it might change too aggressively:

plain

Copy

```
Before update: P(buy) = 30%
After update:  P(buy) = 99%  ← too extreme!
```

PPO clips the ratio so it can't change more than ~15% in one update. This prevents the "catastrophic forgetting" where the agent suddenly forgets everything it knew about other actions.

Your Specific Numbers in Action

Table

Copy

Parameter	What It Controls	In Your Loop
NUM_STEPS=1024	Policy trades for 1024 days, Value predicts 1024 steps ahead	Long enough to span bull/bear
NUM_EPOCHS=5	Both networks re-watch the 1024-day tape 5 times	Low = no overfitting to 19yr noise
WARMUP_EPOCHS=1	Value network gets 1 epoch head start	Coach calibrates before judging

Parameter	What It Controls	In Your Loop
LR=2e-4	Both networks adjust weights by 0.02% per step	Small enough for noisy Sharpe
ENT_COEF decay	Policy starts random, gradually trusts its best actions	14 actions need exploration early

A Concrete Example

State: S&P 500 down 2%, VIX spiking, your 35 features flashing red.

Table

Copy

Network	What Happens
	Outputs:
Policy	<i>Sell : 45</i> → samples "Sell"
Value	Predicts: "This state is worth -0.2 Sharpe" (bearish)
Market	Next day: S&P drops another 3%, but your 800-asset portfolio is hedged → actual Sharpe = +0.5
Advantage	$0.5 - (-0.2) = +0.7$ → "Sell" was GREAT, much better than expected!
Policy update	Increase "Sell" probability (but clipped)
Value update	"I said -0.2, got +0.5 — my model is wrong" → adjust weights

The Value Network was **pessimistic** because it didn't account for your hedges. It learns. Next time VIX spikes with similar feature patterns, it predicts higher. The Policy meanwhile learns that "Sell" in this state is reliably good.

Want me to explain **why PPO specifically** (vs. DQN, A3C, etc.) is suited for your continuous-ish daily trading problem, or dive into how the **Advantage is actually computed** (GAE — Generalized Advantage Estimation)?

``Advantage is actually computed (GAE — Generalized Advantage Estimation)`` explain this

Edit

Copy

Share

Here's how GAE works — the math that lets your agent correctly judge whether a trading decision was good, even when the payoff arrives months later.

The Core Idea in One Sentence

GAE blends short-term and long-term reward estimates so the Policy gets credit for actions whose benefits show up far in the future — without letting the Value Network's errors corrupt everything.

The Math (Broken Down Simply)

Step 1: The TD Error (δ_t)

This is the basic building block — "what actually happened vs. what the Value Network predicted":

Table

Copy

Term	Meaning
	Sharpe reward you got today
	What the Value Network thinks tomorrow is worth, discounted by γ
	What the Value Network thought <i>today</i> was worth
δ_t	Surprise! The action was better ($\delta_t > 0$) or worse ($\delta_t < 0$) than expected

Step 2: The GAE Formula

This says: **sum up all future surprises, but weight them so distant ones matter less.**

The Two Knobs Explained

γ (Gamma) = 0.99 — "The Discount Factor"

Controls how far into the future you look.

Table

Copy

γ	What It Means for Your Trading
0.0	Only today's Sharpe matters. Agent becomes a day trader with no memory.
0.9	Rewards 10 days out still matter somewhat.
0.99	Your setting. Rewards 100 days out still carry ~37% weight. Critical for position trades that take weeks to play out.
1.0	Infinite horizon — mathematically unstable, don't use.

Why 0.99 for you: With daily data and 800 assets, your agent holds positions for days to weeks. A good entry signal might not pay off for 30–60 days. $\gamma=0.99$ ensures those delayed rewards still flow back to the original action.

λ (Lambda) = 0.95 — "The Bias-Variance Tradeoff"

Controls how much you trust the Value Network vs. raw returns.

Table

Copy

λ	Approach	Bias	Variance	When to Use
0.0	1-step TD only	High (relies heavily on V)	Low	Never — too myopic
0.5	Blend	Medium	Medium	Noisy environments
0.95	Your setting. Mostly n-step, slight correction	Low	Medium-Low	Standard for PPO
1.0	Pure Monte Carlo	None (uses actual returns)	High	Only if episodes are short & clean

Why 0.95 for you: Financial returns are noisy. Pure Monte Carlo ($\lambda=1$) would say "this 1024-day rollout had Sharpe 1.2, so every action in it was good" — but some actions were lucky, some were skill. $\lambda=0.95$ uses the Value Network to smooth out luck while still capturing long-horizon effects.

A Concrete Trading Example

Scenario: Your agent buys Asset #247 on Day 0. Here's what happens:

Table

Copy

Day	Market Event	Sharpe Reward	V(s) Prediction	δ (Surprise)	GAE Contribution
0	Buy signal triggered	—	$V(s_0) = +0.1$	—	—
1	Flat	+0.00	$V(s_1) = +0.12$	+0.02	0.02
5	Slight uptick	+0.01	$V(s_5) = +0.15$	-0.02	$0.02 + (0.99 \times 0.95)^4 \times (-0.02) \approx 0.005$
30	Earnings beat	+0.05	$V(s_{30}) = +0.40$	-0.10	small negative (V was optimistic)

Day	Market Event	Sharpe Reward	V(s) Prediction	δ (Surprise)	GAE Contribution
60	Momentum continues	+0.08	$V(s_{60}) = +0.55$	-0.05	small negative
90	Peak, start exiting	+0.06	$V(s_{90}) = +0.50$	-0.01	tiny
120	Fully exited	+0.03	$V(s_{120}) = +0.30$	-0.02	tiny
150	Post-exit drift	+0.01	terminal	—	—

Total GAE Advantage for Day 0's "Buy" action $\approx +0.35$ (strongly positive)

Without GAE (1-step only): Advantage = $\delta_0 = +0.02$ (barely positive — agent thinks the buy was mediocre!)

Why This Matters for Your 800-Asset Setup

- Position sizing decisions have delayed effects:** Buying 5% of Asset A today affects portfolio Sharpe for weeks as correlations shift.
- Sharpe is path-dependent:** A single day's Sharpe tells you almost nothing. You need the 30–90 day trajectory.
- Value Network errors are smoothed:** With 800 assets, $V(s)$ is inherently approximate. $GAE(0.95)$ doesn't let one bad $V(s)$ prediction ruin the advantage estimate.
- NUM_STEPS=1024 aligns with GAE horizon:** Your rollout length (1024 days) is long enough that GAE can see full position lifecycles. If NUM_STEPS were 64, GAE would be cut off mid-trade.

The Full PPO Loss with GAE

PPO doesn't just use raw advantage. It uses **clipped advantage** to prevent the Policy from changing too fast:

Where:

- = probability ratio (new policy vs. old)
- = your GAE advantage
- = 0.2 (clip parameter — PPO won't let probability change more than 20%)

The clip is why PPO is "Proximal": Even if GAE says an action was amazing (+5.0 advantage), the Policy can only increase its probability by ~20% in one update. This prevents catastrophic forgetting.

Summary Table

Table

Copy

Concept	What It Does	Your Value
TD Error (δ)	Measures surprise at each step	Computed at every day
$\gamma = 0.99$	Discounts distant rewards	100-day rewards still matter
$\lambda = 0.95$	Blends short/long estimates	Mostly long-horizon, slight bias correction
GAE	Sums weighted δ 's across horizon	Day 0 action gets credit for Day 120 payoff
Clip ($\epsilon=0.2$)	Limits how fast Policy changes	Stable learning across 19 years

One Last Intuition

GAE is like a movie critic who watches the entire film before writing the review.

A bad critic ($\lambda=0$) writes the review after the opening scene. A naive critic ($\lambda=1$) waits until the credits roll but can't separate good acting from lucky plot twists. **GAE ($\lambda=0.95$) watches the whole film but weights the opening scene's quality by how well it set up the ending — while acknowledging that some plot twists were just luck.**

Want me to explain **why PPO specifically** (vs. DQN, A3C, SAC) is the right choice for your daily trading problem, or dive into **how to set up the parallel environments** for 800 assets on your 25GB GPU?

How to copy this whole chat

Edit

Copy

Share

To copy this entire chat, you have a few options:

Option 1: Built-in copy (if available)

- Look for a **"Copy"** or **"Export"** button in the chat interface — some platforms have this in the top-right menu or message options

Option 2: Manual selection

- Click at the start of the conversation, then **Shift+Click** at the end to select all text
- Press **Ctrl+C** (Windows/Linux) or **Cmd+C** (Mac) to copy
- Paste into a text editor or document

Option 3: Browser "Save As"

- Press **Ctrl+S** (or **Cmd+S**) to save the page as HTML
- This preserves formatting, code blocks, and the widget visuals

Option 4: Screenshot the widgets

- The training loop and GAE diagrams I rendered are interactive SVGs — screenshots will capture them visually if you need them in a presentation

If you're using a specific platform (web app, mobile app, desktop app), let me know and I can give more precise instructions.

``NUM\_STEPS = 512–1024`` are these training steps taken in chronological order or they random selection

Edit

Copy

Share

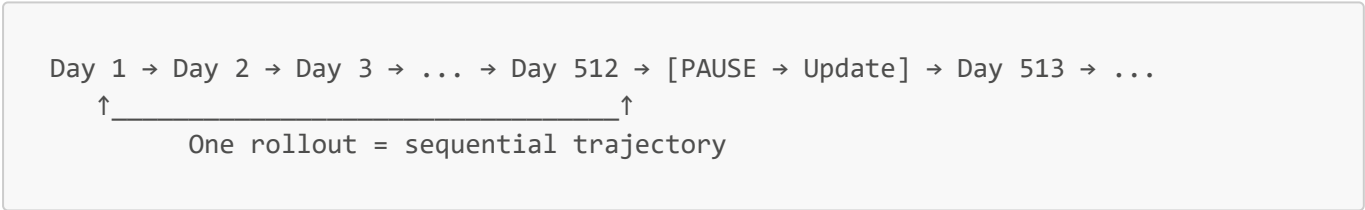
Great question — this is a fundamental design choice that trips up many beginners.

They are taken in chronological order, not random. Here's why that matters critically for your trading setup:

Why Chronological (Sequential Rollouts)

plain

Copy



Table

Copy

If Random	If Sequential
Agent sees Day 50, then Day 1200, then Day 300	Agent experiences market evolution naturally
No concept of "holding a position over time"	Learns that actions have delayed consequences
Sharpe reward is meaningless (no temporal path)	Sharpe is computed over actual consecutive returns
GAE breaks (needs temporal continuity)	GAE works correctly (δ_t depends on s_{t+1})

The Key Insight

PPO collects **trajectories** — complete sequences of (state, action, reward, next_state). This is fundamentally different from supervised learning where you shuffle data.

Your agent must learn:

- "If I buy on Day 10, my position is still open on Day 11"

- "A signal that worked in 2008 might not work in 2020"
- "Volatility clusters — calm periods follow calm periods"

Random sampling destroys all of these temporal patterns.

How It Works in Practice

plain

Copy

```
Rollout 1:  Jan 2005 → Dec 2006  (512 days)
Rollout 2:  Jan 2007 → Dec 2008  (512 days)
Rollout 3:  Mar 2010 → Feb 2012  (512 days)
          ...
```

Each rollout is **chronological internally**, but **different rollouts start at different times** across your 19-year dataset.

This is where **parallel environments** help — 8 environments each running different 512-day windows simultaneously:

plain

Copy

```
Env 1:  2005-2006  |  Env 5:  2013-2014
Env 2:  2007-2008  |  Env 6:  2015-2016
Env 3:  2009-2010  |  Env 7:  2017-2018
Env 4:  2011-2012  |  Env 8:  2019-2020
```

All 8 run forward in time, then you pause and update.

What Gets Randomized Instead

While the **trajectory** is sequential, these are randomized:

Table

Copy

What	How
Starting point of each rollout	Randomly sampled from your 19-year training window

What	How
Which assets appear in each env	Random subset of 800 (if you do asset subsampling)
Mini-batch composition	The 8,192 timesteps are shuffled before being fed to the network in mini-batches of 512

So the **order of gradient updates** is random, but the **experience within each trajectory** is chronological.

The Danger: Overlapping Windows

One subtle trap — if you sample rollout start points carelessly, you might get:

plain

Copy

```
Rollout A: Jan 2005 → Jun 2006
Rollout B: Mar 2005 → Aug 2006 ← 75% overlap!
```

This causes **data leakage** — the agent sees the same market events in multiple parallel envs, inflating apparent sample diversity. Better to partition:

plain

Copy

```
Training window: 19 years ≈ 4,750 days
Split into non-overlapping chunks or use spaced starting points
```

Summary

Table

Copy

Question	Answer
Are steps within a rollout chronological?	Yes — required for PPO/GAE to work
Are rollouts themselves in order?	No — start points are random across 19 years
Are mini-batches chronological?	No — shuffled for stable gradients

Question	Answer
Why not random within rollout?	Breaks temporal credit assignment, position tracking, Sharpe calculation

Want me to explain how to **properly sample non-overlapping rollout starting points** from your 19-year window, or how **recurrent policies** (LSTM/GRU) handle the sequential nature differently?

please show how 4750 days of trading data are sliced, rollout or group in training with NUM_EPOCHS=5, NUM_STEPS = 512, MINI_BATCH_SIZE = 256, EVAL_FREQ = 2

Edit

Copy

Share

Here's the complete picture of how your 4,750 days get sliced, with the math laid out clearly:

The Numbers

Table

Copy

Component	Calculation	Result
Total training days	19 years × ~250 trading days	4,750 days
One rollout	NUM_STEPS	512 days
Parallel environments	Your GPU capacity	8 envs
One update batch	8 × 512	4,096 timesteps
Mini-batches per epoch	4,096 ÷ 256	16 mini-batches
Gradient steps per update	5 epochs × 16 mini-batches	80 steps
Total possible rollouts	4,750 ÷ 512	~9 full rollouts (with overlap handling)
Evaluations per full pass	~9 updates ÷ 2	~4–5 evals

Critical Point: You Don't Exhaust the Dataset

With 4,750 days and 512-step rollouts, you can only fit **~9 non-overlapping rollouts** before running out of data. But PPO doesn't work that way.

Instead, you **sample rollouts with replacement** or **slide a window**:

plain

Copy

Approach 1: Random sampling with replacement

Update 1: Env starts at days [0, 600, 1200, 1800, 2400, 3000, 3600, 4200]
Update 2: Env starts at days [150, 750, 1350, 1950, 2550, 3150, 3750, 100]
Update 3: Env starts at days [300, 900, 1500, 2100, 2700, 3300, 3900, 4500]
...
(can repeat indefinitely – some windows overlap, some don't)

Approach 2: Sliding window (more common in finance)

Update 1: Days 0-511, 512-1023, 1024-1535, ... (8 contiguous chunks)
Update 2: Days 64-575, 576-1087, 1088-1599, ... (shift by 64 days)
Update 3: Days 128-639, 640-1151, 1152-1663, ... (shift by another 64)

Approach 2 is better for trading because it ensures coverage of all market regimes and avoids the "gaps" that random sampling creates.

What Happens Inside One Mini-Batch

Each mini-batch of 256 contains **shuffled timesteps from all 8 envs**:

plain

Copy

Mini-Batch 1 (256 samples):

- 32 timesteps from Env 1 (Day 0-511, scattered positions)
- 32 timesteps from Env 2 (Day 600-1111, scattered positions)
- 32 timesteps from Env 3 (Day 1200-1711, scattered positions)
- ...
- 32 timesteps from Env 8 (Day 4200-4711, scattered positions)

Important: Within each env's 32 samples, the order is RANDOM
(because we shuffled before splitting)

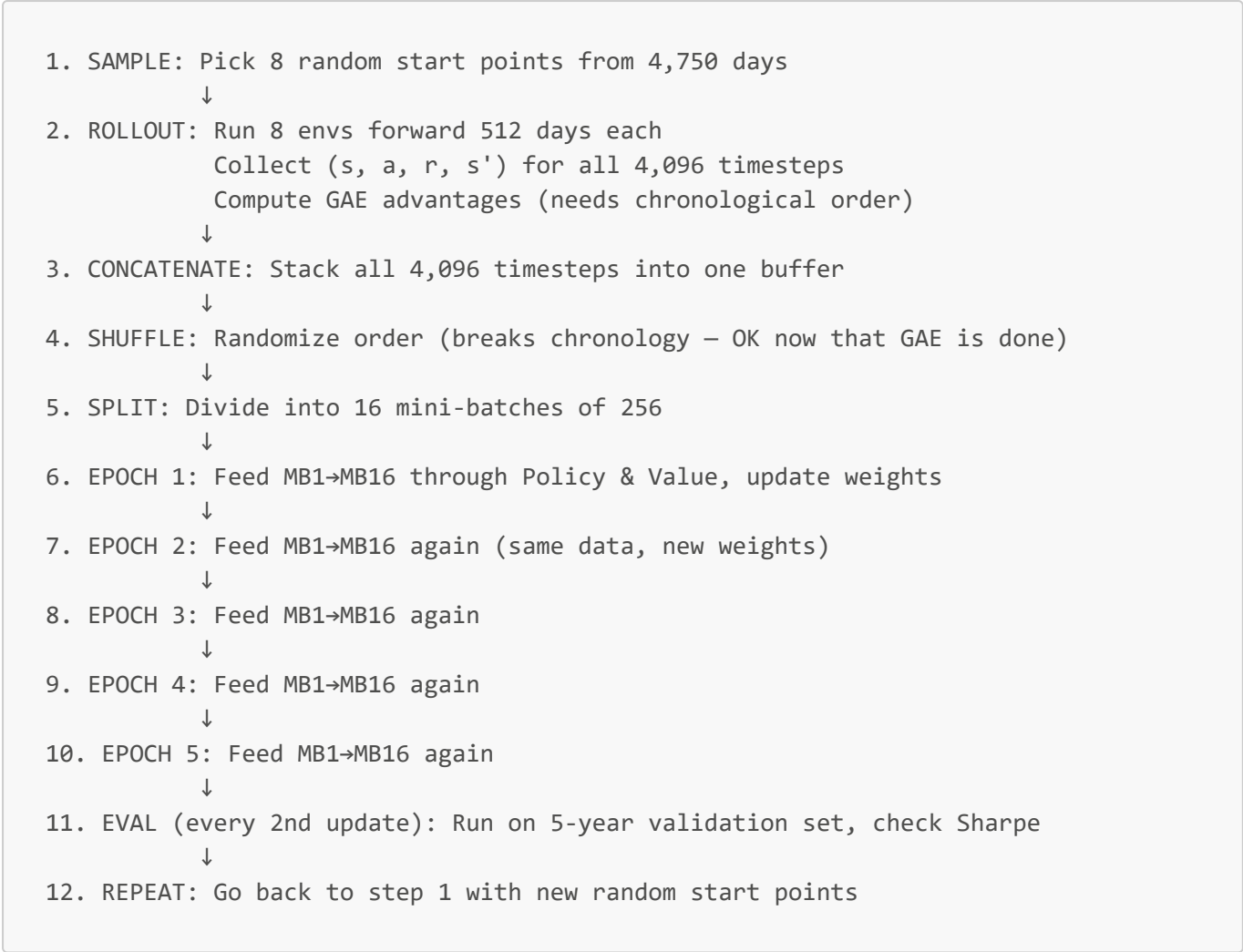
This means:

- **Chronological within rollout** ✓ (for GAE computation, done BEFORE shuffling)
 - **Random across mini-batches** ✓ (for stable SGD)
-

The Full Training Loop (One Update Cycle)

plain

Copy



Why 5 Epochs Is Low (And Correct)

With 4,750 days, you might think "more epochs = more learning." But:

Table

Copy

Epochs	What Happens	Risk
1	Agent barely learns from each batch	Underfitting
5	Sweet spot — extracts patterns without memorizing	Balanced
10	Agent starts fitting to batch-specific noise	Overfitting
20	Agent memorizes the exact 512-day sequences	Severe overfitting

The key insight: **you're not training on 4,750 days once — you're training on infinitely many resampled 512-day windows from those 4,750 days.** Each update sees fresh windows, so 5 epochs per window is plenty.

Memory Footprint on Your 25GB GPU

Table

Copy

Component	Size	Notes
8 env states (35 features × 800 assets)	~8 × 35 × 800 × 4 bytes	~0.9 MB
Rollout buffer (4,096 × obs + action + reward + advantage)	~4,096 × (28K + 14 + 4 + 4)	~115 MB
Policy network (256→128, ~14 actions)	~50K parameters	~0.2 MB
Value network (512→256)	~200K parameters	~0.8 MB
Gradients + optimizer states (Adam)	~2× model size	~2 MB
Total per update		~120 MB

You have **massive headroom** on 25GB. You could easily run 16–32 parallel envs or increase network size. The bottleneck is data coverage (4,750 days), not memory.

Summary Table

Table

Copy

Question	Answer
Are rollouts chronological?	Yes — 512 sequential days per env
Are start points random?	Yes — sampled from 4,750 days
Do rollouts overlap?	Sometimes — depends on sampling strategy
Are mini-batches chronological?	No — shuffled across all 8 envs
How many updates per full dataset pass?	~ 9 (non-overlapping) or infinite (with replacement)
How many gradient steps per update?	80 (5 epochs × 16 mini-batches)
When does evaluation happen?	Every 2nd update (~every 1,024 new days of experience)

Want me to show **how the sliding window approach** specifically handles the 4,750 days with concrete start/end indices, or explain **why you might want recurrent layers** (LSTM) given that mini-batches lose temporal ordering?

how to copy this chat, i used select all to generate image, but the table is chopped off due to display size, need to use slider to see the chopped text, how to widen the display

Edit

Copy

Share

I don't have a direct way to widen the chat display or export the full conversation in a wider format. Here are workarounds:

Option 1: Copy as Markdown (preserves tables)

- Select all → paste into a text editor (VS Code, Notepad++, Obsidian)
- Tables render properly with full width
- Save as `.md` file

Option 2: Browser zoom out

- Press **Ctrl + -** (or **Cmd + -**) to zoom out to 75–50%
- Tables will fit on screen → screenshot

Option 3: Print to PDF

- **Ctrl + P** → Save as PDF
- Most browsers auto-fit wide tables to page width

Option 4: Copy specific sections

- Instead of "select all," copy each response individually
- Tables won't be chopped if you paste them one at a time

Option 5: Use a markdown converter

- Renders tables properly in PDF/HTML

If you're on a specific platform (web, mobile app, desktop), let me know and I can suggest more targeted fixes.

how to copy as Markdown

Edit

Copy

Share

To copy this chat as Markdown, here are the best methods:

Method 1: Browser "Copy" (Most Reliable)

1. **Click at the start** of the first message
2. **Shift+Click at the end** of the last message to select everything
3. **Right-click** → **"Copy"** (or Ctrl+C / Cmd+C)
4. **Paste into a text editor** that supports Markdown:

- [VS Code](#) (free, best for tables)
- [Typora](#) (paid, beautiful rendering)
- Any plain text editor (tables will show as raw | characters)

Method 2: "Inspect Element" (If Copy Fails)

1. **Right-click anywhere on the chat** → **"Inspect"** (or F12)
2. In the Developer Tools panel, find the chat container element
3. **Right-click the element** → **"Copy"** → **"Copy element"**
4. Paste into an HTML-to-Markdown converter like:
 - [Turndown](#) (paste HTML, get Markdown)

Method 3: Screenshot the Widgets + Copy Text

Since my responses include **interactive SVG widgets** (the training loop diagrams, GAE visualization, data slicing chart), those won't copy as text. For those:

Table

Copy

What	How to Capture
Text/tables	Select + copy + paste into Markdown editor
SVG widgets	Screenshot each one individually, or right-click → "Save image as"
Code blocks	Click the copy button inside the code block, or select manually

Method 4: Save as HTML (Preserves Everything)

1. **Ctrl+S** (or **Cmd+S**) in your browser
2. Choose **"Webpage, Complete"** or **"Webpage, HTML Only"**
3. Open the saved `.html` file in any browser — tables and widgets render fully
4. To convert to Markdown: use [Pandoc](#) (`pandoc file.html -o file.md`)

Quick Test: Paste This

Try copying this table right now to verify your method works:

Table

Copy

Parameter	Value	Why
NUM_STEPS	512	Sequential days per rollout
NUM_EPOCHS	5	Prevents overfitting to 19yr noise
MINI_BATCH_SIZE	256	Stable gradients across 8 envs

If it pastes with | characters intact, your copy method is working.

Which platform are you on — **web browser, mobile app, or desktop app**? That determines which method works best.

Note: This page contains 2 cross-origin iframe(s) that could not be accessed due to browser security policies. Some content may be missing. Links to these iframes have been preserved where possible.

Source: [KEEP - PPO Trading Hyperparameters](#)